

RAG Agent 面试八股文

副标题：从向量检索到可信 Agent 问答的通讯版面试准备资料

版本：免费引流通识版

适用对象：

- 想系统准备 RAG、Agent、LLM 应用面试的同学
- 想把 RAG 项目写进简历，但不知道如何讲清楚工程细节的人
- 想从“会调 API”升级到“能解释系统设计、可靠性和评测”的开发者

说明：

这是一份通讯资料，不绑定任何具体项目、代码仓库、训练营系统或公司内部实现。你可以把它当作面试复习提纲、简历表达素材、知识星球赠品、公开分享配套资料使用。

1. RAG 到底解决什么问题？

RAG，全称 Retrieval-Augmented Generation，核心目标不是“让大模型记住更多知识”，而是让大模型在回答时能够使用外部知识库里的可追溯证据。

大模型本身有三个常见问题：

- 知识有截止时间，无法天然知道最新资料。
- 对私有文档、公司内部知识、个人资料没有访问能力。
- 即使回答看起来流畅，也可能没有可靠出处。

RAG 的基本思路是：

用户问题 -> 检索相关文档片段 -> 把证据放进上下文 -> 让模型基于证据回答

面试里不要把 RAG 简化成“向量库 + 大模型”。更好的表述是：

RAG 是一套把外部知识转化为可检索、可引用、可评测证据链的系统工程。向量检索只是其中一环，完整链路还包括数据解析、切块、索引、召回、重排、上下文构造、生成约束、引用校验、拒答和评测。

2. 一句话讲清楚 RAG 项目

30 秒版本：

我做的 RAG 系统不是简单把文档塞进向量库，而是从数据入库、chunk 设计、混合检索、重排、证据约束、引用校验、无证据拒答到评测回归形成一条完整链路。它的目标是让模型回答得更可靠，并且能解释答案来自哪里。

3 分钟版本：

一个完整 RAG 系统通常分为四层。第一层是数据层，负责 PDF、网页、Markdown、数据库等原始资料的解析、清洗和结构化。第二层是索引层，负责 chunk、embedding、向量索引、关键词索引和元数据存储。第三层是推理层，负责 query rewrite、混合召回、RRF 融合、rerank、上下文拼接、生成和引用校验。第四层是工程和评测层，负责状态检查、日志、trace、权限、成本、监控、golden set 和回归测试。面试中真正要讲清楚的不

是“我用了哪个模型”，而是为什么这些环节这样组合，以及它们如何降低幻觉、提升召回和支持后续优化。

3. Chunk 为什么重要？

Chunk 是 RAG 系统里最容易被低估的部分。很多系统效果差，不是因为模型不够强，而是因为文档被切坏了。

常见问题：

- chunk 太小：定义、实验结果、上下文被切断。
- chunk 太大：检索命中了相关文档，但证据不聚焦，模型容易被噪声影响。
- 没有标题和章节信息：片段里出现“this method”“our approach”时，脱离上下文后语义不完整。
- 表格、公式、图片说明被丢失：导致关键证据根本没有进入索引。

面试答法：

Chunk size 本质上是 recall 和 precision 的权衡。太小容易丢上下文，太大容易引入噪声和增加成本。工程上通常会结合文档结构切块，比如标题、章节、段落、表格和图注，同时加一定 overlap，避免关键信息跨边界丢失。真正合理的 chunk 策略应该通过 golden set 做对比，而不是凭感觉定一个固定长度。

高频追问：为什么要加 overlap？

因为很多答案不是刚好完整落在一个切分边界内。Overlap 可以降低定义、结论、步骤被切断的概率。但 overlap 也会增加重复内容、索引体积和 rerank 成本，所以需要通过评测找到合适范围。

高频追问：为什么要保留标题和章节？

单独一个 chunk 往往缺少语境。把文档标题、章节标题或小节路径作为 context prefix，可以让 embedding 更好地表达片段属于哪个主题，也能帮助生成阶段理解证据位置。

4. Embedding 和向量数据库怎么讲？

Embedding 是把文本映射成向量，让语义相近的文本在向量空间里距离更近。向量数据库负责存储这些向量，并在用户提问时做近邻搜索。

但要注意：向量数据库不是 RAG 的全部。

更完整的理解：

- Embedding 擅长语义相似，不一定擅长精确术语、编号、公式、缩写。
- 向量库擅长相似度召回，不适合承载所有业务状态。
- 元数据、权限、文档状态、用户反馈、任务记录通常应该放在结构化存储里。
- 生产系统还要考虑索引重建、版本迁移、过滤条件、备份恢复和数据隔离。

面试答法：

向量数据库解决的是语义召回问题，但 RAG 系统还需要结构化 metadata、权限过滤、关键词召回、重排、引用校验和评测。不能把所有问题都推给向量库。一个可靠系统通常会把向量索引、关键词索引和业务数据库分工设计清楚。

5. Dense Retrieval、Sparse Retrieval 和 Hybrid Retrieval

Dense retrieval 是基于 embedding 的语义检索，适合处理同义表达和概念相近的问题。

Sparse retrieval 常见代表是 BM25 或倒排索引，适合处理关键词、专有名词、缩写、公式、编号、API 名称等精确匹配。

Hybrid retrieval 是把两者结合起来。

为什么需要混合检索？

- 用户问法和原文表述可能不同，需要 dense retrieval。
- 技术文档、论文、代码、法务、医疗等场景里有大量精确术语，需要 sparse retrieval。
- 单一路检索容易有盲区，混合检索能提高召回稳定性。

面试答法：

Dense retrieval 更擅长语义相似，Sparse retrieval 更擅长精确匹配。真实知识库两类问题都会出现，所以我倾向于先做多路召回，再用融合和重排统一排序。这样比单纯依赖向量检索更稳。

6. RRF 是什么？

RRF，全称 Reciprocal Rank Fusion，是一种融合多个排序列表的方法。它不直接比较不同检索器的分数，而是根据排名位置计算融合分数。

直觉理解：

- 一个文档在多个检索器里都排得靠前，应该更可信。
- 一个文档即使只在某一路检索里排得很靠前，也不应该被完全抹掉。
- 不同检索器的分数尺度不同，直接加权可能不稳定。

面试答法：

BM25 分数和向量相似度分数不是同一个尺度，直接加权会比较脆弱。RRF 只看排名，不要求分数可比较，所以适合把 dense、sparse、rewrite query 等多个检索结果融合起来。

7. Rerank 为什么必要？

Embedding retrieval 通常是 bi-encoder：query 和 document 分别编码，再做向量相似度计算。它速度快，适合大规模召回，但相关性判断比较粗。

Reranker 通常是 cross-encoder：把 query 和候选 chunk 放在一起判断相关性。它更准，但更慢。

常见工程流程：

多路召回 -> 候选合并 -> 去重 -> rerank 精排 -> 选择 top-k 进入上下文

面试答法：

Rerank 的价值是把“粗召回”变成“精排序”。向量检索适合从大库里快速找候选，但 top-k 里可能有语义相似但不回答问题的片段。Reranker 会同时看 query 和 chunk，更适合判断是否真正相关。工程上不能全库 rerank，只能先召回候选再精排。

追问：Rerank 会带来什么成本？

它会增加延迟和算力成本。候选窗口越大，质量可能越好，但延迟越高。所以要控制 rerank 的候选数量，并通过评测观察收益是否值得。

8. Query Rewrite 和 HyDE 怎么讲？

真实用户不会总用知识库里的原始措辞提问。Query rewrite 的目标是把用户问题改写成更适合检索的表达。

常见 rewrite 类型：

- 补全缩写和别名。
- 改写成更接近文档语言的检索句。
- 抽取关键词版本，用于 sparse retrieval。
- 生成多个检索变体，提高召回。

HyDE 是 Hypothetical Document Embeddings。它先生成一个“假想答案”或“假想文档片段”，再用这个假想文本去做 dense retrieval。

面试答法：

HyDE 的直觉是，用户问题可能很短，但假想答案更像知识库里的文本。用假想答案做向量检索，有时比直接 embed 问题更容易召回相关内容。但 HyDE 不能作为最终证据，它只参与召回。最终答案仍然必须基于真实检索到的文档片段。

追问：HyDE 会不会引入幻觉？

会有这个风险，所以必须限制它的作用范围。HyDE 只用于生成额外检索 query，不直接进入最终答案的证据链。答案阶段仍然只能使用真实 retrieved chunks，并且要做引用校验和拒答判断。

9. Agentic RAG 和普通 RAG 的区别

普通 RAG 往往是固定链路：

问题 -> 检索 -> 拼上下文 -> 生成

Agentic RAG 更强调系统会根据问题类型、检索结果和证据质量做决策。

常见决策点：

- 是否需要检索？
- 应该查哪个知识源？
- 是否需要 query rewrite？
- 第一轮证据够不够？
- 是否需要 follow-up retrieval？
- 证据不足时是回答、弱答还是拒答？
- 是否需要调用工具生成结构化结果？

面试答法：

Agentic RAG 不是简单给 RAG 加一个 Agent 名字，而是把检索、反思、拒答、工具调用和 trace 做成可控的决策流程。它的目标不是让模型自由发挥，而是让系统围绕证据做更可靠的问答决策。

追问：为什么不用一个通用 Agent 全部自动完成？

通用 Agent 的轨迹可能不稳定，难以评测和回归。对于知识问答，核心不是让 Agent 随机规划，而是让检索、证据判断、引用和拒答稳定可控。关键步骤显式化，系统才更容易调试、评测和上线。

10. Memory 在 RAG 里怎么用？

多轮对话里，系统需要理解用户正在研究什么。但 memory 不能替代证据。

推荐边界：

- Memory 可以用于理解上下文、改写 query、维持用户偏好。
- Memory 不应该直接作为最终答案依据。
- 最终答案仍然应该重新检索知识库，并引用当前检索到的证据。

面试答法：

Memory 解决的是对话连续性，RAG 证据解决的是答案依据。Memory summary 可能压缩、遗漏甚至引入误差，所以我不会把它直接当作事实来源。它可以帮助 query rewrite 和 scope 判断，但最终回答仍然要重新检索，并绑定可追溯引用。

11. 为什么要做拒答？

RAG 系统最危险的不是“答不出来”，而是“没有证据还答得很自信”。

拒答，通常叫 abstain 或 no-answer handling，是可靠 RAG 的关键能力。

拒答触发场景：

- 没有检索到任何相关 chunk。
- 检索分数很低。
- top chunks 互相矛盾。
- 用户问题超出知识库范围。
- 用户要求模型推断没有证据支持的结论。

面试答法：

对知识库问答来说，错误回答比拒答更危险。拒答机制的目标不是让系统少回答，而是在证据不足时不冒充确定。工程上可以在 LLM 生成前根据检索数量、分数、覆盖度和 citation 可用性做判断，证据不足就返回 no evidence 或 weak evidence。

追问：拒答会不会影响用户体验？

会，所以拒答不能只说“不知道”。更好的做法是解释证据不足的原因，并给出下一步建议，比如补充文档、换一种问题、展示当前检索到的弱证据，或者建议用户限定范围。

12. Citation Validation 为什么重要？

引用不是装饰，而是 RAG 系统可信度的核心。

常见错误：

- 模型生成看似真实的引用编号，但并不存在。
- 引用了真实文档，但不支持答案里的 claim。
- 多个 chunk 被拼接后，答案里出现了没有证据支持的新结论。

基础防线：

- Prompt 中要求只能引用给定证据。
- 生成后检查引用 id 是否来自本轮检索结果。
- 对不存在或格式异常的引用进行清理或标记。
- 对关键 claim 做更细粒度的证据映射。

面试答法：

Prompt 约束不能完全防止模型编造引用，所以需要生成后校验。最基础的是检查答案里的引用是否来自本轮 retrieved set；更进一步可以做 statement-level attribution，判断每个关键 claim 是否真的被对应证据支持。

13. 多模态 RAG 怎么讲？

很多知识不只存在于纯文本里，尤其是 PDF、论文、报告、财报、手册和技术文档。

多模态 RAG 常见对象：

- 图片
- 图表
- 表格
- 扫描件
- 公式
- 页面布局

常见方案：

- 1 文本化：用 OCR、表格解析、图像理解模型把视觉内容转成文本摘要，再进入普通 RAG。
- 2 多向量：文本、图片、表格分别建索引，检索时多路召回。
- 3 实时视觉问答：回答阶段把图片交给多模态模型理解。

面试答法：

多模态 RAG 的关键不是简单把图片丢进向量库，而是要让图表里的信息变成可检索、可引用、可评测的证据。实践中常见做法是在入库阶段结合 caption、上下文和视觉模型生成结构化摘要，再把摘要和原始位置绑定。这样既能复用文本检索链路，也方便做引用和评测。

追问：图表摘要会不会丢信息？

会，所以要保留原始图表位置、caption、页码和周边上下文。摘要用于召回和初步理解，但高风险场景可以在回答阶段回看原始图表，或者要求人工确认关键数字。

14. RAG 评测应该怎么做？

只看一次 demo 答案，是最不可靠的评测方式。

RAG 评测至少分四层：

层级	关注点	常见指标
Retrieval eval	证据有没有找对	recall@k, MRR, hit rate
Citation eval	引用是否真实、是否支撑答案	citation precision, citation coverage
Answer eval	答案是否正确、完整、无幻觉	exact match, semantic score, human review
Claim eval	关键语义点是否覆盖且有证据	claim recall, grounded claim recall

面试答法：

我不会只靠人工看一个问题的回答来判断 RAG 优化是否有效。更合理的是固定 golden set，把问题分成 factual、method、compare、summary、ambiguous、no-evidence 等类型，分别看 retrieval recall、citation precision、claim coverage、abstain precision 和 latency。这样才能知道优化到底提升了哪一层。

追问：LLM-as-judge 能不能用？

可以用，但建议作为辅助指标。LLM judge 适合评估语义相似、答案完整性和表达质量，但它本身也可能不稳定，所以关键 gate 还是应该尽量依赖可复现的 retrieval、citation 和规则化 claim 指标。

15. Golden Set 怎么设计？

Golden set 是 RAG 系统的回归测试集。

好的 golden set 不应该只包含简单问题。

建议覆盖：

- 事实问题：能从单个片段找到明确答案。
- 方法问题：需要解释机制或流程。
- 对比问题：需要跨多个片段或多个文档比较。
- 总结问题：需要综合多个证据。
- 模糊问题：问题表达不完整，需要系统澄清或保守回答。
- 无证据问题：知识库里没有答案，应该拒答。
- 错误前提问题：问题本身包含错误假设。

每条样本可以包含：

- question
- expected document

- expected chunks
- must contain
- must not contain
- answer notes
- category
- difficulty

面试答法：

Golden set 的价值是让优化可回归。比如我改了 chunk size 或 query rewrite，不能只看某个 demo question 变好了，而要看整体 retrieval recall、no-evidence 表现、citation precision 和 latency 有没有退化。

16. 如何判断一次 RAG 优化是否有效？

不要只说“答案看起来更好了”。

更严谨的判断方式：

- 1 固定测试集。
- 2 固定评测命令和指标。
- 3 对比优化前后的指标。
- 4 分析 gain 和 harm。
- 5 检查典型失败样本。
- 6 观察延迟和成本变化。

面试答法：

我会先固定 golden set，再比较优化前后的 recall@k、MRR、citation precision、claim recall、abstain precision、latency 和成本。如果是 query rewrite 改动，还会看哪些问题被改好了，哪些原来能命中的被改坏了，避免 query drift。一个优化只有在整体指标和关键案例上都说得通，才算真正有效。

17. Agent 工程化应该关注什么？

很多 Agent demo 看起来很炫，但难以上线，原因是缺少工程边界。

可靠 Agent 系统需要关注：

- Tool allowlist：能调用哪些工具。
- 参数校验：工具输入是否安全、完整、合法。
- 状态管理：多轮任务如何记录进度。
- Trace：每一步为什么这样做。
- Recursion limit：防止无限循环。
- Timeout：防止工具卡死。
- Fallback：外部依赖失败时怎么降级。

- Permission：用户是否有权访问数据。
- Observability：日志、指标、错误、成本。

面试答法：

我不会把 Agent 理解成“让模型自己想办法”。工程里更重要的是给 Agent 明确工具边界、状态、停止条件、错误处理和可观测 trace。这样系统才可调试、可评测、可上线。

18. Agent Tool 封装怎么讲？

Tool 是 Agent 能够调用的外部能力。它可以是检索、问答、文件解析、数据库查询、网页搜索、报告生成、发邮件，也可以是某个业务系统接口。

面试里不要把 tool 讲成“一个函数”。更准确的说法是：

Tool 是给模型使用的受控 API。它不仅包含执行逻辑，还包含名称、用途说明、参数 schema、权限边界、错误返回、审计信息和可评测契约。

一个好的 tool 至少要考虑：

- 名称清晰：模型能从名字判断什么时候该调用。
- 描述准确：说明适用场景、输入要求和不适用边界。
- 参数 schema 严格：必填字段、枚举值、默认值、长度限制要明确。
- 输出结构稳定：不要只返回一段自然语言，最好返回结构化 JSON。
- 错误可恢复：失败时返回错误类型、原因和建议，而不是直接抛异常。
- 权限可控：工具内部也要检查用户、租户和资源权限。
- 可审计：保留 trace id、tool name、输入摘要、输出摘要和耗时。
- 可评测：能写单测、契约测试和端到端测试。

常见 tool 类型：

Tool 类型	解决什么问题	面试重点
Ingest / Build	把资料解析、切块、入库	状态机、失败恢复、幂等
Search / QA	检索证据并回答问题	证据边界、引用、拒答
Compare / Section	对比多个对象或查看局部内容	参数约束、范围控制
Knowledge / Wiki	查询沉淀后的概念或摘要	不能替代原始证据
Discovery	根据主题发现候选资料	候选不等于最终证据
Export / Deliver	生成报告、引用、PPT、文档	引用一致性、格式验证
Feedback / Proactive	收集反馈、订阅、提醒	闭环数据和用户边界

高频追问：Tool 和普通函数有什么区别？

普通函数是给代码调用的，tool 是给模型调用的。模型不能阅读源码，只能看 tool name、description 和 schema，所以 tool 设计要更强调语义清晰、参数约束、错误可解释和调用后可审计。

高频追问：为什么 tool 输出最好结构化？

因为结构化输出更容易被后续步骤消费，也更容易评测和审计。如果 tool 只返回一段自然语言，模型可能误解状态、证据或错误原因。结构化字段可以明确区分 answer、evidence、citations、status、error 和 next_action。

19. Skill、Tool、Subagent 有什么区别？

很多 Agent 项目会同时出现 tool、skill 和 subagent。它们不是一个东西。

概念	本质	适合解决的问题
Tool	可执行能力	查库、检索、生成文件、调用外部系统
Skill	一套操作说明或领域流程	告诉 Agent 如何按步骤完成某类任务
Subagent	带角色、上下文和工具集合的专门 Agent	把复杂任务委派给更聚焦的执行者

可以这样理解：

- Tool 解决“能做什么”。
- Skill 解决“应该怎么做”。
- Subagent 解决“由谁在什么边界里做”。

面试答法：

我会把 tool、skill 和 subagent 分层设计。Tool 是稳定 API，负责具体动作；skill 是任务方法论，指导模型如何使用工具；subagent 是职责收束，把某类任务放到专门上下文里执行。这样比把所有提示词、所有工具和所有状态都塞进一个主 Agent 更清晰。

为什么需要 skill？

- 复杂任务需要固定步骤，比如先检查状态、再读取资料、再生成结果、最后验证。
- 光靠 system prompt 太泛，模型容易漏步骤。
- Skill 可以按需激活，减少主上下文负担。
- Skill 可以沉淀团队方法论，让同类任务更稳定。

为什么需要 subagent？

- 主 Agent 不应该同时承担所有角色。
- 专门任务需要专门工具和约束。
- 可以减少上下文污染。
- 可以限制工具可见范围。
- 可以让主 Agent 只负责分派和整合结果。

高频追问：为什么不把所有工具都暴露给主 Agent？

工具越多，模型误调用的概率越高，tool schema 也会占用上下文。更好的方式是按任务阶段或角色暴露工具，必要时由主 Agent 委派给 subagent，或者用延迟工具发现机制按需打开工具。

20. Agent Runtime 和 Middleware 怎么讲？

一个能上线的 Agent 系统，不能只是一个 while loop。它需要运行时和中间件。

Runtime 负责：

- 线程和会话状态。
- 工具注册和调用。
- 子任务和 subagent 管理。
- 事件流和流式输出。
- 文件上传和工作区。
- 记忆、摘要和上下文预算。
- 日志、trace 和审计。
- 错误处理和安全边界。

Middleware 负责横切能力。它和 prompt 的区别是：prompt 是软约束，中间件是工程约束。

常见中间件：

中间件	作用
Auth / Session	校验用户身份，注入 user context
Rate Limit	控制请求频率，防止滥用
Metrics	统计 QPS、延迟、错误率、token、成本
PII Scrub	对敏感信息做脱敏或阻断
Uploads	管理用户上传文件及可见范围
Memory	抽取和注入长期/短期记忆
Summarization	压缩长对话，控制上下文长度
Deferred Tool Filter	按需暴露工具，减少误调用
Loop Detection	检测重复调用和无效循环
Tool Error Handling	捕获工具异常并返回可恢复错误
Sandbox	限制文件、网络或代码执行边界
Subagent Limit	限制并发子任务，防止失控
Skill Activation	按显式指令或任务类型激活 skill

面试答法：

Prompt 可以提醒模型不要乱做，但不能真正保证系统安全。中间件可以在模型调用前后、工具绑定前后、工具执行前后做硬性控制，比如过滤工具、校验权限、注入文件上下文、检测循环、记录 token 成本和处理工具错误。这是 Agent 从 demo 走向工程系统的关键。

高频追问：为什么 runtime status 很重要？

Agent/RAG 项目经常失败在环境和依赖，比如模型 key、embedding、向量索引、数据库、文件目录、外部服务。Runtime status 能把这些依赖状态显式暴露出来，同时避免泄露 secret。它能显著降低调试和交付成本。

21. Agent 工作台和产品闭环怎么讲？

只做一个聊天框，很难体现 Agent 项目的工程能力。更完整的 AI 应用应该有工作台和产品闭环。

典型 workflow：

发现候选资料 -> 入库构建 -> 检索问答 -> 查看 trace 和引用 -> 生成知识沉淀 -> 收集反馈 -> 进入评测和优化

常见页面或模块：

- Discovery：从主题发现候选资料，并解释为什么选中或跳过。
- Knowledge Builder：展示 fetch、parse、chunk、embed、index、summary 等构建阶段。
- QA：基于证据回答，并展示 citations、chunks、trace、abstain 状态。
- Wiki / Knowledge：把高频概念沉淀成可复用知识页。
- Feedback：收集好评、差评、错误类型和用户备注。
- Inbox / Subscription：订阅主题、接收更新、形成主动提醒。
- Deliverables：生成报告、摘要、PPT、文档、引用清单或其他交付物。

面试答法：

一个真实 AI 应用不应该只有“问一句答一句”。我会把数据构建、问答、引用、反馈、订阅和交付物生成做成产品闭环。这样用户能看到系统状态，开发者也能从反馈和 hard cases 反推下一轮评测和优化。

UI 状态也是工程能力：

- loading：告诉用户当前在解析、检索、生成还是导出。
- empty：没有数据时解释下一步该做什么。
- error：失败时显示可恢复原因。
- success：结果、引用、trace 和下一步动作可读。
- retry：允许用户重试失败阶段，而不是全流程重跑。

高频追问：Discovery 候选能不能直接作为最终答案证据？

不建议。Discovery 只说明“可能相关”，通常来自标题、摘要或外部 metadata。最终答案应该基于已经入库、解析、切块、索引并被本轮检索命中的证据。否则容易把候选信息和真实证据混在一起，破坏引用可信度。

22. 从 RAG 到 Agent 答复应该怎么评测？

如果项目包含 Agent tool、skill、subagent 和工作台，只测 retrieval recall 不够。

更完整的分层评测：

层级	评测重点
单元层	解析、切块、检索、引用校验等函数是否正确
Retrieval 层	gold document / gold chunk 是否被召回
Citation 层	引用是否来自本轮证据，是否支撑 claim
Answer 层	答案是否正确、完整、无明显幻觉

层级	评测重点
Abstain 层	无证据问题是否拒答，弱证据是否保守
Tool 契约层	tool 是否注册、schema 是否稳定、错误是否结构化
Routing 层	Agent 是否在正确问题上调用正确工具
Subagent 层	委派是否合理，subagent 是否遵守证据边界
Gateway 层	API 是否鉴权、是否传递 user scope、是否隐藏 secret
Conversation E2E	多轮对话里是否保持上下文，又不让 memory 污染证据
成本和延迟	token、模型调用、rerank、视觉处理和导出成本是否可控

面试答法：

如果系统已经有 Agent 工具层，就不能只测 RAG 召回。我会同时测 retrieval、citation、claim、abstain、tool contract、tool routing、subagent 行为和 gateway 权限。这样才能从“证据有没有找到”一直追到“Agent 最终答复是否正确调用工具、是否遵守证据边界、是否可审计”。

Tool routing 怎么测？

- 该检索时是否调用检索工具。
- 该入库时是否调用构建工具。
- 该拒答时是否没有乱调用生成工具。
- 该导出时是否调用交付物工具。
- 无权限或无数据时是否返回明确错误。
- 多轮追问时是否保留上下文但重新验证证据。

23. 生产化 RAG/Agent 要考虑什么？

本地 demo 和生产系统差别很大。

生产化关注点：

- 用户级数据隔离：A 用户不能检索到 B 用户数据。
- 权限过滤：检索阶段就要过滤无权限文档。
- 索引版本：文档更新后如何重建和回滚。
- 备份恢复：向量库、元数据、原始文件都要能恢复。
- 成本统计：按用户、任务、模型统计 token 和调用成本。
- 监报告警：延迟、错误率、空召回率、拒答率、成本异常。
- 安全：API key、PII、prompt injection、工具权限。
- CI gate：核心评测集不能在合并后退化。

面试答法：

RAG 上线不只是把 demo 部署到服务器。真正生产化要处理权限、租户隔离、索引迁移、评测回归、监控、成本、备份和安全。尤其是权限过滤必须进入检索链路，否则模型可能拿到用户无权访问的证据。

24. 高频面试问答 70 题

1. RAG 和微调有什么区别？

RAG 适合把外部知识动态接入模型，知识更新快、可追溯、能引用来源。微调更适合改变模型行为风格、格式遵循、特定任务能力。知识密集且频繁更新的场景优先考虑 RAG；稳定模式或专用能力可以考虑微调。

2. 为什么 RAG 不能只靠向量检索？

向量检索擅长语义相似，但对专有名词、缩写、数字、公式、代码符号不一定稳定。完整 RAG 还需要关键词检索、融合、重排、metadata filter、引用校验和评测。

3. Chunk size 怎么选？

看任务和文档结构。原则是既要保留完整语义，又要避免噪声过多。一般通过 golden set 比较不同 chunk size、overlap 和 context prefix，而不是凭经验一次定死。

4. 为什么需要 overlap？

Overlap 能降低答案跨 chunk 边界时丢失上下文的概率。但 overlap 过大会增加重复、索引体积和成本，所以需要评测平衡。

5. 为什么要做 metadata？

Metadata 支持过滤、排序、权限、引用、调试和重建。例如文档 id、章节、页码、时间、作者、用户 id、来源类型都不应该只藏在文本里。

6. BM25 和 embedding 检索有什么区别？

BM25 基于词项匹配，适合精确关键词。Embedding 检索基于语义相似，适合同义表达和概念匹配。两者互补。

7. RRF 为什么比直接加权更稳？

因为不同检索器的分数尺度不同，直接加权需要校准。RRF 只依赖排名，更适合融合异构检索结果。

8. Rerank 和向量检索有什么区别？

向量检索通常是快速粗召回，rerank 是对候选做精排。Rerank 更准但更慢，不能直接对全库使用。

9. Query rewrite 有什么风险？

可能发生 query drift，把用户问题改写到偏离原意的方向。所以要保留原始 query，并评估 rewrite gain 和 harm。

10. HyDE 的风险是什么？

HyDE 生成的假想答案可能带有幻觉。它只能用于召回辅助，不能作为最终证据。

11. 什么是 no-evidence？

当知识库里没有足够证据支持回答时，系统应该明确说明证据不足，而不是让模型凭常识硬答。

12. 为什么拒答是高级能力？

因为可靠系统不仅要知道什么能答，也要知道什么不能答。错误回答在知识库问答里通常比拒答更危险。

13. Citation validation 能完全防幻觉吗？

不能。它能防止伪造引用或引用不存在的 chunk，但不能保证每一句话都被充分支持。更进一步需要 claim-level attribution。

14. 什么是 claim-level evaluation？

把参考答案拆成多个关键 claim，分别检查每个 claim 是否被模型覆盖，以及是否有证据支持。

15. 为什么不能只用 LLM judge？

LLM judge 有用，但可能受 prompt、模型版本和上下文影响。它适合作辅助评估，不适合成为唯一硬指标。

16. RAG 的常见失败模式有哪些？

解析失败、chunk 切坏、embedding 不匹配、召回错误、rerank 失效、上下文过长、模型忽略证据、引用不真实、拒答阈值不合理、权限过滤缺失。

17. 如何 debug 一个错误答案？

从 trace 看：原始问题是什么，rewrite 生成了什么，召回了哪些 chunk，rerank 后 top-k 是什么，证据是否足够，prompt 里放了什么，最终引用是否支撑答案。

18. 为什么要做 trace？

没有 trace 就只能看最终答案，无法定位问题出在检索、重排、生成还是引用。Trace 是 RAG 调试和评测的基础。

19. Agentic RAG 比普通 RAG 强在哪里？

它能围绕证据做决策，比如是否 rewrite、是否二次检索、是否拒答、是否调用工具。重点是可控决策，而不是自由发挥。

20. Agent 的工具调用如何防失控？

使用工具白名单、参数 schema、权限校验、超时、最大步数、审计日志和敏感操作确认。

21. Memory 会污染答案吗？

如果直接把 memory 当事实来源，会。更好的做法是让 memory 只辅助上下文理解和 query rewrite，最终答案仍然依赖实时检索证据。

22. 多轮 RAG 为什么难？

因为系统要区分用户是在追问前文、切换主题、纠正前提还是要求跨文档综合。历史太少会断上下文，历史太多会引入噪声。

23. 多模态 RAG 难在哪里？

图表、表格、公式和页面布局不容易转成干净文本。即使转成摘要，也可能丢失关键数字和结构关系。

24. 图表应该怎么入库？

常见做法是保留图片位置、caption 和周边上下文，再用 OCR 或视觉模型生成结构化摘要，作为可检索文本入库。

25. 表格 RAG 怎么处理？

简单表格可以转成 Markdown 或行级文本；复杂表格需要保留表头、单位、行列关系和来源位置。有时 SQL/结构化查询比纯向量检索更合适。

26. Prompt 在 RAG 里有多重要？

Prompt 很重要，但不是全部。Prompt 控制回答格式和证据使用，检索和证据质量决定模型能不能答对。

27. 如何防 prompt injection？

不要让检索内容直接获得系统权限。需要区分系统指令和文档内容，对工具调用做权限控制，对高风险指令做过滤和审计。

28. RAG 如何做权限控制？

权限过滤应该进入检索阶段。不能先检索全库再让模型判断，因为模型一旦看到无权数据，就已经泄露了。

29. 用户级数据隔离怎么做？

文档、chunk、向量 payload、查询过滤、日志和缓存都要带用户或租户 scope。不能只在 UI 层做隔离。

30. RAG 的成本主要在哪里？

Embedding、rerank、LLM 生成、多轮检索、长上下文、视觉模型调用和评测调用都会产生成本。

31. 如何降低成本？

缓存 embedding 和检索结果，控制 top-k 和 rerank 窗口，缩短上下文，按场景选择模型，失败前置判断，避免无证据问题也调用大模型。

32. RAG 为什么需要 fallback？

外部模型、向量库、reranker、parser 都可能失败。Fallback 可以让系统降级返回部分结果或错误说明，而不是直接崩溃。

33. 什么是 hard case？

Hard case 是系统失败或边界表现差的样本。把用户反馈和失败案例沉淀为 hard case，可以驱动下一轮 golden set 和优化。

34. 如何设计 no-answer 样本？

包含无关领域、知识库外问题、错误前提、时间敏感问题和权限外问题。它们用来验证系统是否知道自己不知道。

35. 如何评估召回？

看 gold document 或 gold chunk 是否出现在 top-k 中，常用 recall@k、MRR、hit rate。更细可以看 citation chunk 是否被召回。

36. 如何评估答案？

可以看 must contain、must not contain、claim coverage、人工评分和 LLM judge。高风险场景要重视证据支撑，而不是只看语义像不像。

37. 为什么评测要分类？

不同问题类型失败原因不同。事实题、对比题、总结题、无证据题不能混在一个总分里，否则很难知道该优化哪里。

38. CI golden gate 是什么？

在代码合并前自动跑核心评测，防止检索、引用或拒答能力退化。它不一定覆盖全部生成评测，但应该覆盖低成本、可复现的关键指标。

39. RAG 和搜索引擎有什么区别？

搜索引擎返回文档列表，RAG 基于检索证据生成答案。RAG 更重视证据选择、上下文构造、生成约束和引用可信度。

40. RAG 和知识图谱有什么关系？

知识图谱适合结构化实体关系和可解释推理，RAG 适合非结构化文档问答。两者可以结合，用图谱做实体约束和关系扩展，用 RAG 处理原文证据。

41. 什么时候不该用 RAG？

纯闲聊、固定规则、简单数据库查询、强实时计算、严格事务流程不一定需要 RAG。不要为了用 RAG 而用 RAG。

42. 什么场景适合 Agent？

需要多步骤决策、工具调用、状态跟踪、动态计划和错误恢复的场景更适合 Agent。单轮简单问答不一定需要。

43. Agent 和 workflow 怎么取舍？

如果流程稳定、规则明确，用 workflow 更可靠。如果任务开放、路径不确定，可以引入 Agent，但要加边界和 trace。

44. 如何解释“可控 Agent”？

可控 Agent 不是让模型无限自主，而是在工具、状态、权限、步数、错误处理和观测上有明确约束。

45. 如何写 RAG 简历 bullet？

不要只写“接入向量数据库和大模型”。应该写清楚场景、技术决策、可靠性机制和评测结果，例如“设计混合检索与重排链路，引入引用校验和无证据拒答，并用 golden set 做回归评测”。

46. 面试官问项目亮点，怎么答？

从完整链路讲：数据入库、混合检索、证据约束、拒答、评测和工程化。不要只讲 UI 或模型调用。

47. 面试官问项目不足，怎么答？

诚实讲边界：解析复杂文档仍有误差，评测集需要扩展，线上权限和成本统计需要加强，多模态证据仍需要更细粒度评测。然后给出优化优先级。

48. 如果线上召回突然变差怎么办？

检查数据是否入库成功、embedding 版本是否变化、索引是否完整、metadata filter 是否过严、query rewrite 是否漂移、reranker 是否异常、文档权限是否过滤掉了结果。

49. 如果答案有引用但还是错怎么办？

说明 citation existence 不等于 citation support。需要检查引用 chunk 是否真正支持 claim，并考虑 statement-level citation 或 claim-level eval。

50. 一句话总结可靠 RAG 的核心？

可靠 RAG 的核心不是让模型更会说，而是让答案和证据建立可审计、可评测、可回归的关系。

51. Agent tool 封装最重要的是什么？

最重要的是契约稳定。Tool 要有清晰名称、准确描述、严格参数 schema、结构化输出、错误类型、权限检查和审计 trace。否则模型即使能调用，也很难稳定使用。

52. Tool schema 应该怎么设计？

参数尽量显式，少用自由文本大字段。能用枚举就用枚举，能限制长度就限制长度，能拆字段就不要混在一个 prompt 里。输出也要结构化，区分 status、data、evidence、error 和 next_action。

53. Tool 和 skill 有什么区别？

Tool 是可执行能力，skill 是使用能力的方法论。Tool 回答“能做什么”，skill 回答“遇到某类任务应该怎么做”。

54. Skill 有什么价值？

Skill 可以把固定流程沉淀下来，比如先检查状态、再读取资料、再执行工具、最后验证结果。它减少主 prompt 负担，也让同类任务更稳定。

55. Subagent 有什么价值？

Subagent 用来收束职责。主 Agent 负责理解任务和分派，subagent 负责某个专业任务。这样可以限制工具范围、减少上下文污染，也更容易做行为评测。

56. 为什么不把所有工具都暴露给主 Agent？

工具越多，误调用概率越高，tool schema 也会占用上下文。按任务阶段、角色或 skill 按需暴露工具，通常比一次性暴露全部工具更稳。

57. 什么是 deferred tool exposure？

它指一开始不把所有工具都给模型，而是根据任务需要逐步开放工具。这样可以减少上下文开销，也降低模型误选工具的概率。

58. Tool routing 怎么评测？

看 Agent 是否在正确场景调用正确工具，并在不该调用时保持克制。例如无证据问题应该拒答，导出任务应该调用导出工具，知识库为空时应该提示先入库。

59. Tool error handling 为什么重要？

工具可能超时、参数错误、权限不足或外部依赖失败。好的系统会返回结构化错误和可恢复建议，而不是让整个 Agent 崩溃。

60. 中间件和 prompt 的区别是什么？

Prompt 是软约束，模型可能不遵守；中间件是工程约束，可以在工具绑定、模型调用、工具执行和响应返回前后做硬性检查。

61. Runtime status 应该显示什么？

显示关键依赖是否可用，比如模型、索引、数据库、文件存储、外部服务、权限配置和后台任务状态。注意不要显示 API key、token 或敏感路径。

62. Gateway 和 Agent runtime 有什么区别？

Gateway 负责 HTTP API、认证、路由、限流、指标和用户上下文。Agent runtime 负责模型调用、工具执行、线程状态、记忆、subagent、trace 和运行控制。

63. 同步回答和流式回答怎么取舍？

同步接口简单，适合短任务和脚本调用。流式接口体验更好，适合长答案、长工具链和可视化 trace，但要处理断连、事件顺序、错误事件和前端状态。

64. 上传文件进入 Agent 后要注意什么？

要限制文件类型、大小、路径和可见范围。上传文件应属于当前用户或线程，不能被其他用户检索到，也不能让模型通过文件内容执行越权指令。

65. Sandbox 的作用是什么？

Sandbox 用来限制 Agent 对文件、网络、代码执行或系统资源的访问。它不是为了影响模型回答，而是为了限制工具造成的真实副作用。

66. 为什么工作台比单聊天框更有工程价值？

工作台能展示构建状态、检索证据、trace、反馈、订阅、导出和错误恢复。单聊天框只能展示最终答案，很难证明系统是可观察、可调试、可优化的。

67. Discovery 候选为什么不能直接当最终证据？

候选只代表可能相关，通常来自标题、摘要或外部 metadata。最终答案应该基于已入库并被检索命中的证据，否则引用链路不干净。

68. Feedback 如何进入优化闭环？

用户反馈应该被结构化记录，转成 hard cases，再经过人工或规则审核进入 golden set。这样反馈才会真正推动检索、拒答和生成策略优化。

69. 交付物生成如何保证引用一致？

报告、PPT、文档、引用清单都应该复用同一套证据和 citation 格式。不能在 QA 里有引用，导出时又重新编一套来源。

70. Agent 答复的端到端评测看什么？

看它是否正确理解任务、选择工具、传参、处理错误、检索证据、生成答案、保留引用、遵守权限、记录 trace，并在证据不足时拒答。

25. 简历表达模板

基础版：

构建了一个面向业务文档的 RAG 问答系统，支持文档解析、chunk 切分、embedding 索引、向量检索和基于证据的答案生成。

进阶版：

设计并实现了一个可评测的 RAG 问答链路，结合 dense retrieval、sparse retrieval、RRF 融合和 rerank 提升召回质量，并通过 citation validation、no-evidence abstain 和 golden set 回归降低幻觉风险。

Agentic 版：

构建了 Agentic RAG 工作流，引入 query rewrite、多轮 evidence reflection、工具调用、状态 trace 和拒答机制，使系统能够根据证据质量动态决定继续检索、回答或拒答。

Agent 工程版：

设计 Agent 工具封装和运行时边界，覆盖 tool schema、skill 激活、subagent 委派、middleware guardrails、runtime status、流式事件、错误降级和工具调用审计。

产品闭环版：

将 RAG 能力产品化为 AI 工作台，覆盖资料发现、知识库构建、证据问答、trace 展示、反馈采集、订阅提醒和交付物生成，并通过 hard cases 驱动评测集迭代。

生产化版：

推进 RAG 系统工程化落地，覆盖权限过滤、用户级数据隔离、状态检查、监控指标、成本统计、备份恢复和 CI golden gate，提升系统可运维性和回归稳定性。

26. 面试避坑回答

错误说法：

我用了向量数据库，所以能回答文档问题。

更好说法：

向量数据库只解决语义召回的一部分。完整 RAG 还需要文档解析、chunk 设计、稀疏检索、结果融合、重排、上下文构造、引用校验、拒答和评测。

错误说法：

Agent 就是让大模型自己思考。

更好说法：

工程里的 Agent 更像带状态、工具和决策边界的工作流。关键不是“自由思考”，而是如何控制工具调用、记录 trace、处理失败和保证结果可评测。

错误说法：

RAG 可以解决幻觉。

更好说法：

RAG 可以降低幻觉，但不能天然消除幻觉。只有把证据检索、生成约束、引用校验、拒答和评测结合起来，才能把幻觉风险变成可观察、可控制的问题。

错误说法：

评测就让大模型打分。

更好说法：

LLM judge 可以作为辅助，但 RAG 评测应该先看可复现指标，比如 retrieval recall、citation precision、no-evidence precision 和 claim coverage，再结合人工或 LLM judge 看表达质量。

错误说法：

我把一个函数注册成 tool，所以就是 Agent 项目。

更好说法：

Tool 注册只是第一步。真正的 Agent 工程还要设计 tool schema、权限边界、错误处理、状态管理、trace、tool routing 评测和用户级数据隔离。

错误说法：

Skill 就是一个更长的 prompt。

更好说法：

Skill 更像可复用的任务流程说明，它应该告诉 Agent 什么时候使用、按什么步骤执行、需要哪些输入、如何验证结果，以及遇到失败如何处理。

错误说法：

子 Agent 越多越智能。

更好说法：

Subagent 的价值是职责收束，不是堆数量。每个 subagent 都应该有清晰边界、工具集合、输入输出契约和评测方式。

27. 自测清单

如果你准备把 RAG 或 Agent 项目写进简历，至少要能回答下面这些问题：

- 我为什么需要 RAG，而不是直接问大模型？
- 我的文档是怎么解析和清洗的？
- chunk size、overlap、metadata 是怎么设计的？
- 为什么不能只用向量检索？
- dense、sparse、RRF、rerank 分别解决什么问题？
- query rewrite 和 HyDE 的收益与风险是什么？
- 系统如何判断证据不足？
- 引用是怎么校验的？
- 如何证明一次优化真的有效？
- golden set 覆盖了哪些问题类型？
- 多轮对话里 memory 和 evidence 的边界是什么？
- Agent 工具调用如何限制权限和步数？

- tool、skill、subagent 的区别是什么？
- tool schema 如何设计，错误如何返回？
- 为什么不能把所有工具都暴露给主 Agent？
- 中间件和 prompt 的边界是什么？
- runtime status 应该暴露哪些信息，不应该暴露哪些信息？
- 工作台相比单聊天框多体现了哪些工程能力？
- Discovery 候选为什么不能直接当最终证据？
- feedback 如何进入 hard case 和评测集？
- 如果某个依赖失败，系统怎么降级？
- 线上如何做用户级数据隔离？
- 如何监控延迟、错误率、空召回率和成本？
- 项目当前最大局限是什么，下一步会优先优化什么？

28. 最后总结

面试里讲 RAG 和 Agent，不要停留在“我接了一个模型和一个向量库”。

真正有含金量的表达应该围绕七件事：

- 1 数据如何变成可检索证据。
- 2 检索如何兼顾语义召回和精确匹配。
- 3 生成如何被证据约束。
- 4 系统如何在证据不足时拒答。
- 5 Tool、skill、subagent 如何形成可控 Agent 能力。
- 6 Runtime、中间件和工作台如何支撑工程化交付。
- 7 优化如何通过评测回归证明有效。

记住一句话：

RAG Agent 的核心不是把更多上下文和更多工具塞给模型，而是建立一条从问题到证据、从证据到工具调用、从工具调用到答案、从答案到评测的可信链路。